STEP 2 - Data Dictionary and Leakage Audit

Hotel Booking Cancellation Project | Data Exploration series

What this PDF is: a fact sheet for every one of the 32 columns (type, missing values, distinct values) and a verdict on each: is it safe to use as a feature? All numbers come from the notebook **STEP_2_data_dictionary.ipynb** in this same folder, which you can re-run cell by cell.

Files for this step

File	What it is
STEP_2_data_dictionary.ipynb	The notebook that produced every number and chart here (run top to bottom).
STEP_2_column_dictionary.csv	The full 32-row dictionary table (open in Excel if you like).
figures/step2_missing_values.svg	Chart 1: missing values per column.
figures/step2_suspect_cancel_rates.svg	Chart 2: cancellation rate for the leakage suspects.
figures/step2_single_feature_auc.svg	Chart 3: single-column predictive power (leakage screen).

Libraries used in this step

Library	Version	Why we use it here
pandas	2.3.3	Column profile, value counts, crosstab, groupby cancellation rates, date arithmetic.
numpy	2.5.2	Numeric helpers for the chart positions.
matplotlib	3.11.1	Draws the two charts. Saved as SVG only (one image format for the whole folder).
scikit-learn	1.9.0	DecisionTreeClassifier + cross_val_score + StratifiedKFold for the leakage screen (cross-validated ROC-AUC).
nbformat + nbclient	5.11.1 / 0.11.0	Build and execute the notebook so the saved outputs are real.
svglib	2.3.0	Installed in this step so the SVG charts can sit inside the PDF as sharp vector graphics.
reportlab	5.0.1	Builds this PDF.

1. Why build a data dictionary before cleaning anything?

Cleaning without a plan is blind. The dictionary tells us, column by column, **what exists, what is broken, and what must never be used**. It also forces the most important question in applied ML: *"Would I know this value at the moment I need the prediction?"* A column that fails this test is leakage, and a model built on leakage looks brilliant in testing and fails in real use.

For every column we record four facts:

- **dtype** - what pandas thinks the column is (number or text).
- **missing** - how many values are empty, and what % of the 119,390 rows that is.
- **n_unique** (also called **cardinality**) - how many different values the column has. 2 values means a flag, 10 means a category, 8,879 means a continuous number.
- **verdict** - USABLE, NEEDS CARE, LEAKAGE, or TARGET, with the reason written down.

2. Missing values

Only four columns contain real blanks:

Column	Missing rows	% of 119,390	Why it matters
company	112,593	94.31%	Almost empty. A typical choice is to reduce it to a "has a company" yes/no flag, or drop it.
agent	16,340	13.69%	Missing probably means "no travel agent" - which is information, not just a gap.
country	488	0.41%	Small gap. Many ways to handle (new category, most frequent value).
children	4	0.00%	Only 4 rows. This is why the column is float, as Step 1 predicted.

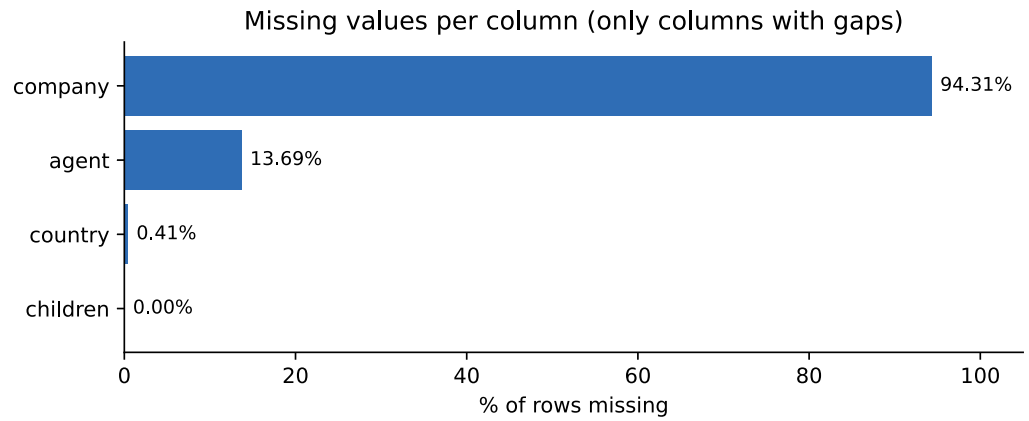


Chart 1 - Missing values per column. Source: notebook Part B.

Important: agent and company are IDs, not quantities

agent has 333 distinct values and **company** has 352. They are *identifiers* (like a roll number), not measurements. The average of agent 9 and agent 229 is meaningless. Two consequences: we must treat them as categories, and we must **not** fill missing values with the mean (which would invent a non-existent agent number).

Hidden missing values: "Undefined"

pandas counts only true blanks. A text value such as **Undefined** is present in the file but means "unknown" - a **disguised missing value**. We scanned every text column and found them:

Column	Rows with "Undefined"	Comment
meal	1,169	Codes in the data: BB 92,310 | HB 14,463 | SC 10,650 | Undefined 1,169 | FB 798. The dataset documentation says Undefined and SC both mean "no meal package", so they may be merged later (decision for the preprocessing step).
market_segment	2	Tiny. Likely safe to treat as missing or fold into the most common group.
distribution_channel	5	Tiny, same treatment.

3. Numeric summary - what the min and max reveal

describe() gives the mean, median (50%), min and max. Impossible or extreme values stand out in min and max long before any chart. Highlights:

Column	Min	Median	Mean	Max	What to notice
lead_time	0	69	104.0	737	Mean well above median: a long right tail of far-ahead bookings.

Column	Min	Median	Mean	Max	What to notice
adr	-6.38	94.58	101.83	5400	A NEGATIVE price and a price of 5,400 per night. Both need investigation.
adults	0	2	1.86	55	Rows with zero adults; 55 adults in one booking is not a normal stay.
children	0	0	0.10	10	4 missing values.
babies	0	0	0.01	10	10 babies in one booking looks like an error.
stays_in_weekend_nights	0	1	0.93	19	Long stays exist.
stays_in_week_nights	0	2	2.50	50	50 weeknights is unusual.
days_in_waiting_list	0	0	2.32	391	Almost always 0, a few huge values.
previous_cancellations	0	0	0.09	26	Mostly 0, heavy tail.

Mean vs median: when the mean is noticeably larger than the median, a few very large values are pulling it up (a right-skewed column). The median is robust to outliers; the mean is not. We will use this idea again in preprocessing and when we choose the regression metric.

These oddities (negative or huge adr, zero adults, zero-night stays) are queued for **Step 3: Data Quality**. We only note them here.

4. Leakage audit - with evidence

The test applied to every column: "At the moment the booking is made (when we would run the prediction), would I really know this value?" If the answer is no, it cannot be a feature. We tested the suspects with data.

4.1 reservation_status is the answer written in words

reservation_status	is_canceled = 0	is_canceled = 1	Total
Canceled	0	43,017	43,017
Check-Out	75,166	0	75,166
No-Show	0	1,207	1,207
Total	75,166	44,224	119,390

The cross-table is perfectly clean: Check-Out is always 0; Canceled and No-Show are always 1. Knowing reservation_status gives a 100% correct answer, which is exactly why it must be excluded.

New fact we did not have before: in `is_canceled`, **No-Show counts as cancelled** (1,207 rows, about 1%). So the target means "the room was not used by this guest", not strictly "the guest clicked cancel". We keep this definition in mind when we explain the project. (Our Step 1 PDF said the status says "Canceled" for exactly the rows where is_canceled = 1; the precise statement is "Canceled or No-Show". That sentence is corrected.)

4.2 reservation_status_date: a date that exists only because the outcome already happened

What the column is: the date on which the reservation_status was last updated. That is the key. The date is not something the guest chooses when booking; the hotel system stamps it *when the outcome happens*.

Two bookings, both for arrival on 25 Dec 2016, 3 nights, booked on 1 March 2016:

	Booking A	Booking B
What happens	Guest cancels on 30 Oct 2016	Guest stays and leaves on 28 Dec 2016
reservation_status	Canceled	Check-Out
reservation_status_date	2016-10-30 (BEFORE the arrival date)	2016-12-28 (AFTER arrival; = departure date)
Did this date exist on 1 March, when we would predict?	NO - created on 30 Oct	NO - created on 28 Dec

Why this leaks the answer: put the status date next to the arrival date. If the status date is *before or on* the arrival date, the booking was cancelled (a cancellation always happens before the guest would have arrived). If it is *after* arrival, the guest stayed. A model can read the answer straight off these two columns without learning anything about guests.

We tested this with a one-line rule and no machine learning: "if reservation_status_date minus arrival date is 0 or negative, predict cancelled; otherwise predict not cancelled".

Method	Accuracy
Always predict "not cancelled" (baseline)	62.96%
The one-line date rule (no model at all)	**99.41%**

A rule you can write in one line gets 99.41%. In real use at booking time, the date would not exist, so the deployed model could not use it and would collapse to a far lower accuracy. That gap between test score and real-life score is exactly what leakage causes.

The supporting facts from the data:

- For **Check-Out** rows the status date equals the departure date (arrival + nights) in **99.96%** of rows - recorded after the stay.
- For **Canceled** rows the date is before arrival in **97.95%** of rows; median **56 days before arrival**. It is literally the cancellation date.

Everyday analogy: predicting whether a parcel was delivered by looking at the "delivered on" stamp. The stamp exists only because delivery happened, so using it to predict delivery is circular. Verdict: **LEAKAGE**.

4.3 assigned_room_type: a strong signal that is still leakage

We compared reserved_room_type with assigned_room_type. When they differ ("room changed"), the cancellation rate collapses:

Group	Rows	Cancellation rate
Room not changed	104,473	41.56%
Room changed	14,917	5.38%

Why this is leakage - the mechanism: the hotel assigns the actual room when the guest *arrives*. A booking that is cancelled never reaches check-in, so its assigned room simply stays equal to the reserved one. A changed room therefore almost proves the guest showed up. The model would learn "room changed means not cancelled", but at booking time that information does not exist yet.

Honest note on strength: this is a *partial* leak. Only about 12.5% of bookings have a changed room, so on its own the "room changed" column scores a modest ROC-AUC of 0.585. We exclude it because of *when and why* the value is created (timing and mechanism), not because it is overwhelmingly strong.

Signal is not the same as leakage. A huge gap in cancellation rate only says the column is *related* to the target. What makes it leakage is **timing** (or that it encodes the outcome). Always ask when the

value becomes known.

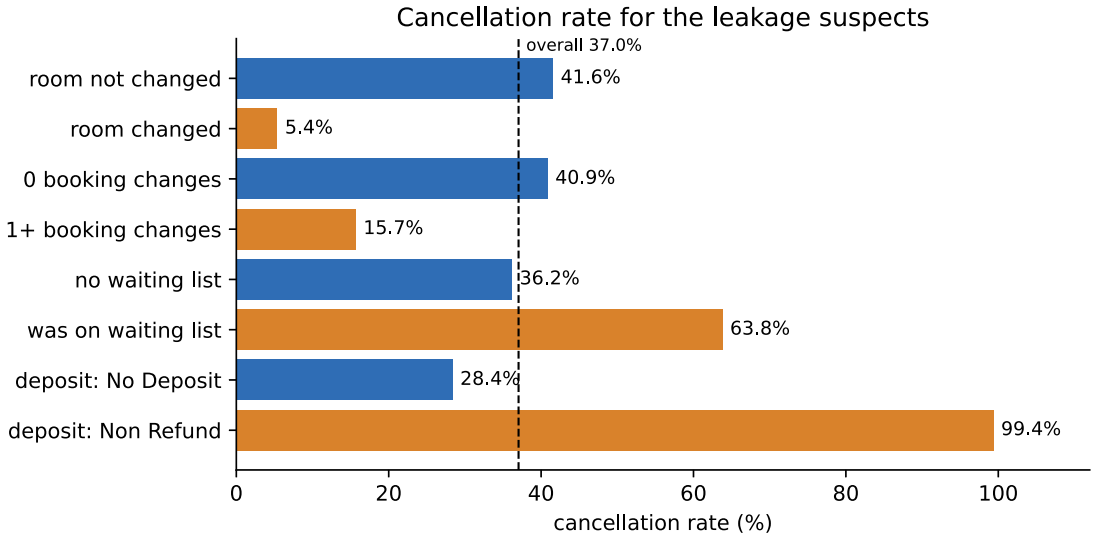


Chart 2 - Cancellation rate by group for the leakage suspects (dashed line = overall 37.04%). Source: notebook E4.

4.4 The suspects we cannot decide by logic alone

Column / group	Rows	Cancellation rate	Reading
booking_changes = 0	101,314	40.85%	Most bookings never change.
booking_changes = 1	12,701	14.23%	Changed bookings cancel far less.
booking_changes = 2	3,805	20.13%	
booking_changes = 3 or more	1,570	16.56%	
days_in_waiting_list = 0	115,692	36.19%	
days_in_waiting_list > 0	3,698	63.79%	Waiting-list bookings cancel much more.

- **booking_changes** is the *final* count. At the moment of booking it is always 0. Changes happen later, and a guest who cancels early has little time to change anything. So the final count partly reflects the outcome. Verdict: leakage suspect.
- **days_in_waiting_list** is known only after the waiting period ends, so the timing is borderline. It is a real operational feature if the prediction is run at confirmation time, but not at the instant of booking.
- We will not drop them blindly. In the **ablation experiments** we train with and without each one and explain the score change.

4.5 A counter-intuitive pattern: deposit_type

deposit_type	Rows	Cancellation rate
No Deposit	104,641	28.38%
Non Refund	14,587	99.36%
Refundable	162	22.22%

Common sense says a guest who paid a non-refundable deposit would be *less* likely to cancel. The data says the opposite: 99.36% cancel. We do not know the reason yet and we will not invent one. Possible directions to investigate in EDA: the hotel, the year, the market segment and the country of those bookings, and whether this is how the hotel records cancelled bookings. Until explained, this column is **NEEDS CARE**: very strong, but we must understand it before trusting it.

4.6 Second pass: a data-driven re-audit of all 31 columns

Judging by logic alone can miss something. So we added a second method. **Single-column screen:** for each column we trained a tiny decision tree (depth 5) that sees *only that column* and measured cross-validated **ROC-AUC** (5 folds). ROC-AUC is 0.5 for random guessing and 1.0 for perfect prediction (the full definition comes in the metrics PDF). A column that alone gets a near-perfect score is almost certainly leaking.

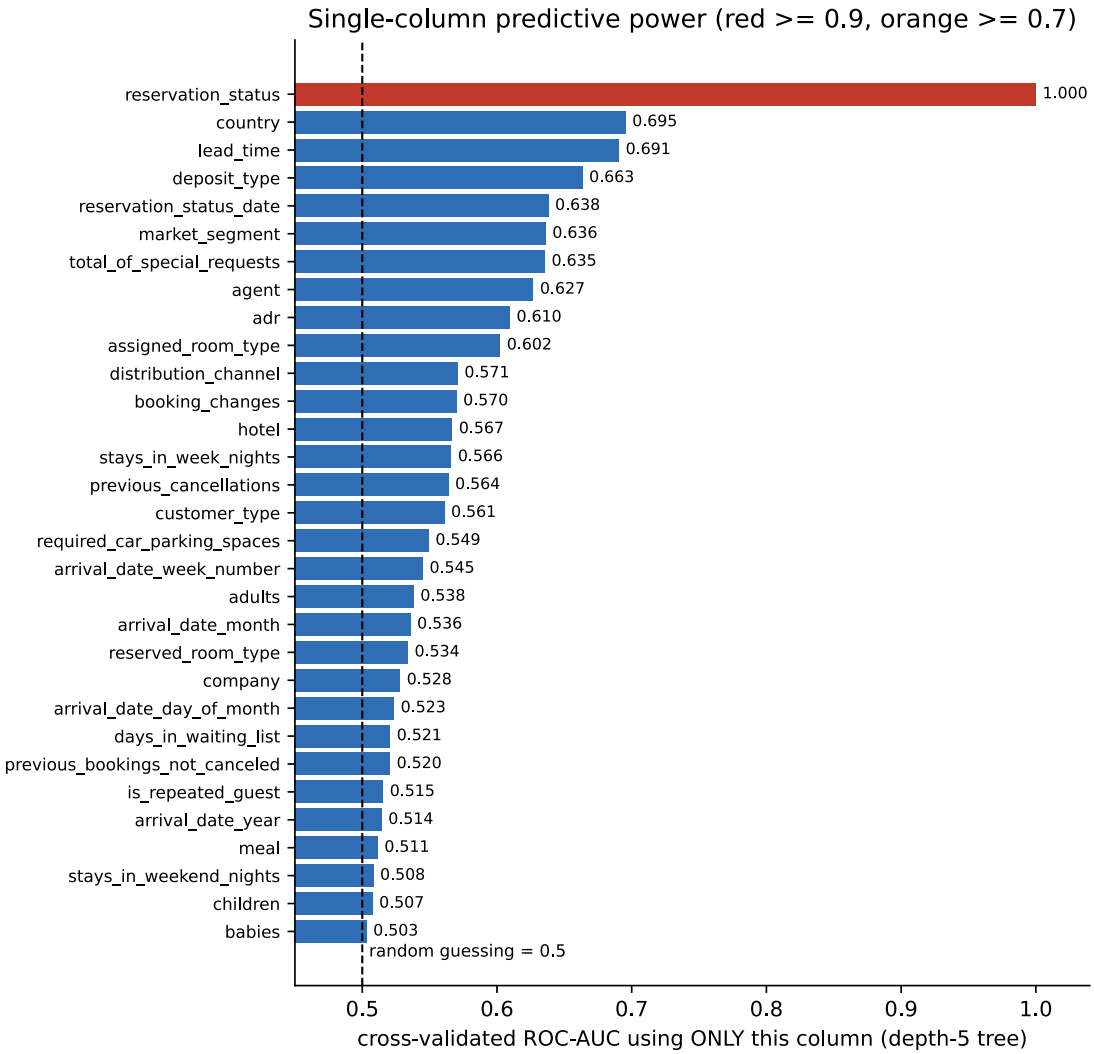


Chart 3 - Single-column ROC-AUC for all 31 columns (dashed line = random guessing). Source: notebook Part F.

Column	Single-column AUC	Comment
reservation_status	1.0000	Perfect: the screen catches this one immediately.
country	0.6954	Legitimate: where the guest comes from is known at booking.
lead_time	0.6905	Legitimate: known at booking.
deposit_type	0.6634	Strong because of the Non Refund oddity (see 4.5).
reservation_status_date	0.6381	LOW - yet we proved it leaks (see below).
total_of_special_requests	0.6355	Moderate; timing suspect.
assigned_room_type	0.6023	Low alone; leak is partial (see 4.3).
all other columns	0.50 - 0.63	Weak alone. No other column leaks perfectly by itself.

The blind spot of the screen: leakage hidden between two columns

The screen scored reservation_status_date at only 0.638. Does that make it safe? No. The leak appears when it is *combined* with the arrival date. We built `gap = status_date - arrival_date`

and ran the same screen:

Derived column	AUC	Reading
gap (status date minus arrival date)	**0.9998**	Near perfect - the date leak lives in the relationship between two columns.
room_changed (reserved room != assigned room)	0.5848	Modest strength (partial leak), excluded for timing and mechanism.

Lesson: a low single-column score does not prove a column is safe. Leakage can hide in combinations of columns. That is why we use **two methods together**: (1) timing logic ("when is this value created?") and (2) data screens.

A new leak found in the re-audit: required_car_parking_spaces

Parking spaces required	Rows	Cancellation rate
0	111,974	39.49%
1	7,383	**0.00%**
2 or more	33	**0.00%**

Out of 7,416 bookings that required parking, **not a single one was cancelled**. Real human behaviour never produces exactly zero. The explanation is the same kind as with the room: parking is recorded only for guests who actually arrive. A cancelled booking never gets a parking record, so it stays at 0. In my first pass I marked this column "needs care", which was too lenient; the data corrected me. Verdict: **LEAKAGE**.

A reusable trick: look for groups with a cancellation rate of exactly 0% or 100% and enough rows. A perfect group in real-world behaviour almost always means the column is recorded after the outcome.

Remaining columns checked

Check	Result	Reading
total_of_special_requests	0 requests: 47.72% cancel | 1: 22.02% | 2: 22.10% | 3+: 16.82%	Gradual pattern, not a perfect split. Requests can be added after booking, so timing is suspect: kept as NEEDS CARE and settled by ablation.
previous_cancellations	Any previous cancellation: 91.64% cancel (6,484 rows) vs 33.91%	Very strong but legitimate: guest history is known before this booking, and past behaviour predicts future behaviour.
is_repeated_guest	Repeated guests: 14.49% cancel vs 37.79%	Legitimate and plausible.
adr = 0 share	2.32% of non-cancelled vs 0.48% of cancelled	Zero-price rows are mostly stays that happened (Step 3 will explain them). No leakage signal.
mean adr	Non-cancelled 99.99 | Cancelled 104.96	Similar; adr is quoted at booking. Fine to use (outliers handled in Step 3).

Is the audit complete?

All 31 non-target columns have now been examined by both methods. Final leakage list: **reservation_status, reservation_status_date, assigned_room_type, required_car_parking_spaces**. Suspects settled later by ablation: **booking_changes, days_in_waiting_list, total_of_special_requests**. **Honest limit:** an audit can find leakage but can never prove there is none. Two more safeguards remain: the ablation experiments, and a rule for the final model - if its score looks too good to be true, investigate before celebrating.

5. The data dictionary - all 32 columns

Verdict colours: green = USABLE, yellow = NEEDS CARE, red = LEAKAGE, blue = TARGET. Totals: 10 usable, 17 need care, 4 leakage, 1 target.

Column (type)	Missing	Unique	Verdict	Reason
hotel object	0	2	USABLE	Chosen at booking. Two values.
is_canceled int64	0	2	TARGET	Classification target. 1 includes both Canceled (43,017) and No-Show (1,207).
lead_time int64	0	479	USABLE	Known at booking. Skewed, max 737 days; check outliers in EDA.
arrival_date_year int64	0	3	NEEDS CARE	Only 2015-2017. A model cannot generalise to a year it never saw; decide how to split by time.
arrival_date_month object	0	12	USABLE	Known at booking. Text month names need ordering/encoding.
arrival_date_week_number int64	0	53	NEEDS CARE	Overlaps with month: redundant information, check correlation.
arrival_date_day_of_month int64	0	31	USABLE	Known at booking, probably weak alone.
stays_in_weekend_nights int64	0	17	NEEDS CARE	Planned stay known at booking, but max 19 and zero-night stays exist (Step 3).
stays_in_week_nights int64	0	35	NEEDS CARE	Planned stay known at booking, but max 50 and zero-night stays exist (Step 3).
adults int64	0	14	NEEDS CARE	Known at booking, but zero-adult rows and max 55 exist (Step 3).
children float64	4	5	NEEDS CARE	4 missing values (stored as NaN, hence float).
babies int64	0	5	USABLE	Known at booking. Max 10 looks odd; check in Step 3.
meal object	0	5	NEEDS CARE	'Undefined' (1,169 rows) is a disguised missing value.
country object	488	177	NEEDS CARE	488 missing; 177 distinct values (high cardinality); PRT is about 41% of rows.
market_segment object	0	8	NEEDS CARE	'Undefined' appears in 2 rows; overlaps with distribution_channel.
distribution_channel object	0	5	NEEDS CARE	'Undefined' appears in 5 rows; overlaps with market_segment.
is_repeated_guest int64	0	2	USABLE	Guest history, known at booking.
previous_cancellations int64	0	15	USABLE	Guest history, known at booking. Heavily skewed; any previous cancellation -> 91.6% cancel (strong but legitimate).
previous_bookings_not_canceled int64	0	73	USABLE	Guest history, known at booking. Heavily skewed.
reserved_room_type object	0	10	USABLE	Chosen at booking time.
assigned_room_type object	0	12	LEAKAGE	Assigned at check-in. Room changed -> 5.4% cancel vs 41.6%: a room can only be reassigned for guests who arrive.
booking_changes int64	0	21	NEEDS CARE	LEAKAGE SUSPECT: final count includes changes made after booking (always 0 at booking time). Test by ablation.
deposit_type object	0	3	NEEDS CARE	Non Refund shows 99.36% cancellation: opposite of common sense. Investigate before trusting.

Column (type)	Missing	Unique	Verdict	Reason
agent float64	16,340	333	NEEDS CARE	13.69% missing; these are IDs (333 distinct), not quantities. Missing may mean "no agent".
company float64	112,593	352	NEEDS CARE	94.31% missing; IDs, not quantities. Likely reduce to a has-company flag or drop.
days_in_waiting_list int64	0	128	NEEDS CARE	LEAKAGE SUSPECT on timing: known only after the waiting period. Waiting>0 -> 63.8% cancel.
customer_type object	0	4	USABLE	Known at booking. Four values.
adr float64	0	8,879	NEEDS CARE	Regression TARGET. As a feature for cancellation it is quoted at booking, but has min -6.38 and max 5400.
required_car_parking_spaces int64	0	5	LEAKAGE	All 7,416 bookings with 1+ spaces have 0 cancellations (exactly 0.00%): parking is only recorded for guests who arrive.
total_of_special_requests int64	0	6	NEEDS CARE	Requests can be added after booking (timing suspect). 0 requests -> 47.7% cancel, 3+ -> 16.8%: a gradual pattern, not a perfect split.
reservation_status object	0	3	LEAKAGE	Canceled/No-Show map exactly to is_canceled = 1. It IS the answer.
reservation_status_date object	0	926	LEAKAGE	For Canceled rows it is the cancellation date; for Check-Out rows it equals departure date (99.96%).

Remember: these verdicts are our current judgment. NEEDS CARE does not mean "bad" - it means "a decision is required, and we must be able to defend it". Every such decision will be recorded with its reason in the preprocessing step.

6. Interview questions from this step (with model answers)

Q1. What is data leakage and how did you detect it in your project?

Answer: Leakage is when a feature contains information that would not be available at prediction time or that directly encodes the target. I applied a timing test to each column: "would I know this at booking time?" I confirmed it with data: a cross-table showed reservation_status maps exactly onto is_canceled, and reservation_status_date equals the departure date on 99.96% of check-outs.

Q2. assigned_room_type is strongly related to cancellation. Why not keep it - it makes the model better?

Answer: It makes the model better in testing only. The room is assigned at check-in, so a changed room almost proves the guest arrived (5.4% cancel vs 41.6%). A cancelled booking never reaches check-in. At booking time the value does not exist, so the deployed model would lose that signal and perform much worse than the test score promised.

Q3. Is every feature that is strongly related to the target leakage?

Answer: No. A strong relationship is a signal; leakage is about timing or encoding the outcome. lead_time is strongly related to cancellation and is perfectly legitimate because it is known at booking. The question is always "when is this value known?"

Q4. Explain why reservation_status_date leaks the target.

Answer: The date is stamped by the hotel system when the outcome happens: the cancellation date for cancelled bookings, the departure date for stays. Compared with the arrival date, "on or before arrival" means cancelled and "after arrival" means stayed. A one-line rule on that difference scores 99.41% accuracy with no model, but at booking time the date does not exist yet.

Q5. You ran a single-column leakage screen and reservation_status_date scored only 0.64 AUC. Is it safe?

Answer: No. Leakage can hide in the relationship between columns. Combined with arrival date as a gap, the AUC is 0.9998. A single-column screen finds obvious leaks (like reservation_status at 1.0) but not those that need a second column, so I use timing logic together with data screens.

Q6. How can you spot leakage in a feature like required_car_parking_spaces?

Answer: Look for groups with a cancellation rate of exactly 0% or 100% that have many rows. All 7,416 bookings with parking had 0 cancellations, which real behaviour does not produce. It means the value is recorded only for guests who arrive, so it exists only after the outcome.

Q7. Can you prove a dataset has no leakage left?

Answer: No. An audit can only find leakage. I reduce the risk with timing logic for every column, data screens, ablation experiments, and a sanity rule: if the final score looks too good to be true, investigate.

Q8. deposit_type "Non Refund" has a 99% cancellation rate. What do you do?

Answer: I would not trust it or drop it blindly. I would first investigate by hotel, year, market segment and country to find out whether it is a real behaviour or a recording artifact, then run ablation (with vs without) and explain the score change. Decide only after I can explain it.

Q9. agent and company have missing values. Would you fill them with the mean?

Answer: No. They are IDs, so a mean has no meaning and invents a fake agent. I would treat them as categories; missing can be its own category (it likely means "no agent / no company"). For company at 94% missing, a has-company flag or dropping it is reasonable.

Q10. What is a disguised missing value? Give an example from this data.

Answer: A value present in the file that really means "unknown", such as the text "Undefined" in meal (1,169 rows), market_segment (2) and distribution_channel (5). pandas does not count them as missing, so you must scan the category values to find them.

Q11. Why are mean and median of adr different, and what does that tell you?

Answer: Mean 101.83 vs median 94.58, with a maximum of 5,400. A few extreme prices pull the mean up. It signals right skew and outliers, and that the median or a robust metric may describe typical behaviour better.

Q12. Your target includes No-Show as cancelled. Is that correct?

Answer: It is a definition choice: 1,207 rows (about 1%). From a hotel point of view a no-show also leaves the room unused, so grouping makes sense, but it must be stated clearly. If the business cared about early cancellations only, we would define the target differently.

Q13. What is cardinality and why does country (177 values) cause trouble?

Answer: Cardinality is the number of distinct values in a column. One-hot encoding 177 countries creates 177 sparse columns. Common options: group rare countries into "Other", or use frequency/target encoding (carefully, to avoid leakage).

7. Self-check (answer in your own words before Step 3)

- In your own words, with the two example bookings (A and B), why does reservation_status_date reveal the answer?
- Why did the single-column screen give reservation_status_date a low score, and what does that teach you?
- Why is required_car_parking_spaces leakage? What number in the table gave it away?
- Why is assigned_room_type leakage even though it predicts cancellation so well? Explain the mechanism, not just "it is known later".
- What is the difference between a strong signal and leakage? Give one example of each from this data.
- Why must agent and company not be filled with the mean?
- What does "Undefined" in the meal column mean for missing-value counts?
- The target includes No-Show. Why might that matter to the hotel?
- booking_changes is not proven leakage. How will we decide whether to keep it?

What comes next: STEP 3 - Data Quality

We investigate the oddities noted here, with counts and reasons for every decision: adr negative or huge and adr = 0, zero-adult and zero-guest bookings, zero-night stays, extreme lead times, and duplicate rows. We decide what to keep, fix, flag or remove, and write each decision down.